

DevOps Linux Command Cheat Sheet

বাংলা ব্যাখ্যা সহ -- সম্পূর্ণ DevOps গাইড

File System · Permissions · Process · Network · Git · Docker · Kubernetes · SSH · Systemd

এই Cheat Sheet-এ আছে:

- ✓ 90+ Essential Linux Commands
- ✓ প্রতিটি command বাংলায় বিস্তারিত ব্যাখ্যা
- ✓ Real-world DevOps examples
- ✓ Git, Docker, Kubernetes commands
- ✓ Server monitoring & troubleshooting
- ✓ SSH, Cron, Systemd -- সব কিছু

সতর্কতা: DevOps-এ sudo ও rm -rf ব্যবহারের আগে সতর্ক থাকো -- production-এ double-check করো

1. File System Commands

DevOps কাজের ৬০% হলো files নিয়ে কাজ -- logs দেখা, config edit করা, backup নেওয়া। এই commands না জানলে server-এ কাজই করা যাবে না।

Command	কী করে (ব্যাখ্যা)	Example
<code>ls -la</code>	সব files দেখাও -- hidden সহ, permission ও size সহ। l=long format, a=all।	<code>ls -la /var/log/nginx/</code>
<code>ls -lhS</code>	Size অনুযায়ী sort করে দেখাও, human-readable format-এ (KB, MB, GB)।	<code>ls -lhS /var/log/</code>
<code>cd /path</code>	Directory পরিবর্তন করো। <code>cd ~</code> = home-এ যাও, <code>cd -</code> = আগের directory-তে ফিরো।	<code>cd /etc/nginx/conf.d/</code>
<code>pwd</code>	বর্তমানে কোন directory-তে আছো তা দেখাও। Script-এ current path জানতে দরকার।	<code>pwd # /home/ubuntu</code>
<code>mkdir -p</code>	Nested directories একসাথে তৈরি করো। Already exist করলে error দেবে না।	<code>mkdir -p /app/logs/2024/jan</code>
<code>cp -rp src dst</code>	পুরো directory copy করো। -p = permission ও timestamp preserve করো।	<code>cp -rp /app /backup/app_\$(date +%F)</code>
<code>mv old new</code>	File rename বা move করো। Same filesystem-এ instant -- data copy হয় না।	<code>mv app.conf app.conf.bak</code>
<code>rm -rf dir/</code>	Directory মুছো। DANGER: undo নেই! Production-এ path দুইবার check করো।	<code>rm -rf /tmp/old_builds/</code>
<code>find -name '*.log'</code>	পুরো system-এ নাম দিয়ে file খোঁজো। type, size, mtime দিয়ে filter করা যায়।	<code>find /var -name '*.log' -mtime +30</code>
<code>find -size +100M</code>	বড় files খোঁজো। Disk full হলে এভাবে বড় file identify করো।	<code>find / -size +500M -type f 2>/dev/null</code>
<code>tar -czvf bk.tgz dir</code>	Directory compress করে archive তৈরি করো। c=create, z=gzip, v=verbose।	<code>tar -czvf backup_\$(date +%F).tar.gz /etc/</code>
<code>tar -xzvf file.tgz</code>	Archive extract করো। -C flag দিয়ে নির্দিষ্ট directory-তে extract করো।	<code>tar -xzvf app.tar.gz -C /opt/</code>
<code>rsync -avz src dst</code>	Files efficiently sync করো। Changed files-ই transfer হয় -- cp-এর চেয়ে fast।	<code>rsync -avz /app/ user@server:/backup/</code>
<code>ln -s target link</code>	Symbolic link (shortcut) তৈরি করো। নতুন version deploy-এ link update করলেই হয়।	<code>ln -s /opt/app/v2.0 /app/current</code>

Tip: Backup নেওয়ার সময় rsync ব্যবহার করো, cp নয় -- rsync resume করতে পারে এবং শুধু changed files transfer করে।

2. Text Processing & Log Analysis

DevOps engineer-এর প্রতিদিনের কাজ হলো logs analyze করা। Error খোঁজা, pattern বের করা, real-time monitoring -- সব এই commands দিয়ে হয়।

Command	কী করে (ব্যাখ্যা)	Example
<code>cat file.txt</code>	File-এর সব content দেখাও। ছোট config file দেখার জন্য উপযুক্ত।	<code>cat /etc/hosts</code>
<code>less file.txt</code>	বড় file page-by-page দেখাও। q=quit, /=search, n=next match।	<code>less /var/log/syslog</code>
<code>tail -f app.log</code>	Log file real-time follow করো। DevOps-এ সবচেয়ে বেশি ব্যবহৃত command।	<code>tail -f /var/log/nginx/error.log</code>
<code>tail -n 100</code>	File-এর শেষ 100 line দেখাও। সাম্প্রতিক errors দেখতে কাজে লাগে।	<code>tail -n 50 /var/log/app.log</code>
<code>grep 'ERROR' log</code>	File-এ নির্দিষ্ট pattern খোঁজো। -i = case-insensitive, -v = invert।	<code>grep -i 'error warn' /var/log/app.log</code>
<code>grep -c 'ERROR'</code>	Pattern কতবার আছে গণনা করো। Error frequency বোঝার জন্য।	<code>grep -c 'ERROR' /var/log/app.log</code>
<code>grep -A 3 -B 2</code>	Error-এর আগে ও পরের lines দেখাও। Context বুঝতে সাহায্য করে।	<code>grep -A 3 -B 2 'Exception' app.log</code>
<code>awk '{print \$1}'</code>	Column-based data process করো। Nginx/Apache log analyze করতে অসাধারণ।	<code>awk '{print \$1, \$9}' access.log</code>
<code>sed 's/old/new/g'</code>	Text replace করো। -i = file-এই directly replace করো।	<code>sed -i 's/localhost/prod-db/g' config.yml</code>
<code>sort uniq -c</code>	Unique entries গণনা করো। Top IPs বা most common errors বের করতে।	<code>awk '{print \$1}' access.log sort uniq -c sort -rn head -10</code>
<code>wc -l file</code>	File-এ কতটি line আছে গণনা করো। Log size বোঝার জন্য।	<code>wc -l /var/log/app.log</code>
<code>pipe ()</code>	Commands জোড়া লাগাও। এক command-এর output অন্য command-এর input হয়।	<code>tail -f app.log grep -i 'error warn'</code>
<code>xargs</code>	Output কে পরের command-এর argument হিসেবে পাঠাও।	<code>find . -name '*.log' xargs grep 'ERROR'</code>

Real DevOps Use: `tail -f /var/log/nginx/error.log | grep -i '500|502|503' --` এই command দিয়ে real-time-এ server error monitor করা যায়।

3. File Permissions & Ownership

Linux security-র মূল ভিত্তি হলো permissions। Wrong permissions মানে security breach বা application crash। r=read(4), w=write(2), x=execute(1) -- এই তিনটি মনে রাখো।

Command	কী করে (ব্যাখ্যা)	Example
<code>chmod 755 file</code>	Owner: rwx(7), Group: r-x(5), Others: r-x(5)। Web directory, executable scripts।	<code>chmod 755 /var/www/html/</code>
<code>chmod 600 key</code>	Owner শুধু read-write পারবে, অন্য কেউ কিছু না। SSH private key-এর জন্য অবশ্যই।	<code>chmod 600 ~/.ssh/id_rsa</code>
<code>chmod 644 conf</code>	Owner read-write, বাকি সবাই শুধু read। Config files ও web assets-এর জন্য।	<code>chmod 644 /etc/nginx/nginx.conf</code>
<code>chmod +x script</code>		<code>chmod +x deploy.sh && ./deploy.sh</code>

	Execute permission যোগ করো সবার জন্য। Shell script run করার আগে এটা দিতে হয়।	
chown user:group	File-এর owner ও group পরিবর্তন করো। App deploy-এর পর correct ownership দাও।	chown -R www-data:www-data /var/www/
chown -R user dir	Directory ও সব contents-এর ownership একসাথে বদলাও। -R = recursive।	chown -R deploy:deploy /opt/app/
ls -l file	File-এর permission দেখো। Output: -rwxr-xr-x 1 user group size date name।	ls -l /etc/passwd

4. Process Management

Production server-এ কোন process কতটুকু resource নিচ্ছে, কোনটা hang করেছে, কোনটা restart দিতে হবে -- এসব জানাটা DevOps-এ critical।

Command	কী করে (ব্যাখ্যা)	Example
ps aux	সব user-এর সব running process দেখাও। PID, CPU%, MEM%, command সহ।	ps aux grep '[n]ginx'
ps --sort=-%mem	Memory ব্যবহার অনুযায়ী sort করো। Memory leak খুঁজে বের করতে কাজে লাগে।	ps aux --sort=-%mem head -10
top	Real-time CPU ও RAM usage দেখাও। q=quit, k=kill, P=CPU sort, M=mem sort।	top -u www-data
htop	top-এর উন্নত version -- color-coded, interactive। (sudo apt install htop)	htop
kill -15 PID	Gracefully বন্ধ করো। SIGTERM -- process নিজে cleanup করে বন্ধ হয়।	kill -15 \$(pgrep nginx)
kill -9 PID	জোর করে বন্ধ করো। SIGKILL -- process ignore করতে পারে না। Last resort।	kill -9 1234
pkill -f 'name'	Name দিয়ে process kill করো। PID না জেনেই process বন্ধ করা যায়।	pkill -f 'unicorn'
nohup cmd &	Terminal বন্ধ হলেও process চলতে থাকবে। Background-এ long process run-এর জন্য।	nohup python app.py > app.log 2>&1 &
lsof -i :8080	কোন process কোন port use করছে দেখাও। Port conflict debug করতে কাজের।	lsof -i :80 -i :443

5. System Monitoring & Performance

Server কতটা healthy সেটা বোঝার জন্য এই commands। CPU, RAM, Disk, Network -- সব monitor করতে হয় DevOps engineer-কে।

Command	কী করে (ব্যাখ্যা)	Example
<code>df -h</code>	Disk কতটুকু full -- human-readable। Disk full হলে application crash করে।	<code>df -h grep -v tmpfs</code>
<code>du -sh /path/*</code>	কোন folder কতটুকু space নিচ্ছে। Disk full হলে বড় folder খুঁজে বের করো।	<code>du -sh /var/log/* sort -h tail -10</code>
<code>free -h</code>	RAM কতটুকু ব্যবহার হচ্ছে। Available, used, swap সব দেখা যাবে।	<code>free -h</code>
<code>vmstat 1 5</code>	CPU, memory, I/O -- সব একসাথে 1 সেকেন্ড পর পর দেখাও।	<code>vmstat 1 10</code>
<code>iostat -x 1</code>	Disk I/O statistics। %util বেশি হলে disk bottleneck আছে।	<code>iostat -x 1 5</code>
<code>ss -tuln</code>	কোন port কোন service listen করছে দেখাও। Open ports audit করতে জরুরি।	<code>ss -tuap grep LISTEN</code>
<code>uptime</code>	Server কতক্ষণ চলছে ও load average। Load > CPU core count = server overloaded।	<code>uptime</code>
<code>dmesg tail</code>	Kernel messages। Hardware error, disk failure, OOM killer -- সব এখানে লগ হয়।	<code>dmesg -T tail -20 grep -i error</code>
<code>journalctl -u svc</code>	Systemd service-এর logs। -f = follow, --since দিয়ে সময় filter করো।	<code>journalctl -u nginx -f --since '1 hour ago'</code>
<code>watch -n 2 'cmd'</code>	Command প্রতি N সেকেন্ডে run করো। Real-time monitoring-এর সহজ উপায়।	<code>watch -n 2 'df -h && free -h'</code>

6. Network Commands

Microservice architecture-এ network troubleshooting জরুরি। Service-এর মধ্যে connection হচ্ছে কিনা, DNS কাজ করছে কিনা, firewall block করছে কিনা -- এই commands দিয়ে বের করা যায়।

Command	কী করে (ব্যাখ্যা)	Example
<code>ping -c 4 host</code>	Host reachable কিনা check করো। Network connectivity-র প্রথম test।	<code>ping -c 4 8.8.8.8</code>
<code>curl -I https://</code>	HTTP headers দেখাও -- status code, server type। Quick web server check।	<code>curl -I -w '%{http_code}' https://myapp.com</code>
<code>curl -v -X POST</code>	HTTP request-এর সব detail দেখাও। API debugging-এর জন্য সবচেয়ে কাজের tool।	<code>curl -v -X POST -H 'Content-Type: application/json' https://api/ep</code>
<code>wget -O - url</code>	URL থেকে content download করো stdout-এ। Script-এ health check করতে।	<code>wget -qO- http://localhost:8080/health</code>
<code>nslookup domain</code>	DNS resolution check করো। Domain কোন IP-তে point করছে দেখো।	<code>nslookup myapp.com 8.8.8.8</code>

dig domain	DNS-এর বিস্তারিত information। DNS propagation check করতে সেরা।	dig myapp.com +short
traceroute host	Packet কোন path দিয়ে যাচ্ছে দেখাও। Latency কোথায় বাড়ছে identify করতে।	traceroute google.com
nc -zv host port	Port open আছে কিনা check করে। Firewall block আছে কিনা পরীক্ষা করে।	nc -zv redis-server 6379
ip addr show	Network interface ও IP address দেখাও। ifconfig-এর modern replacement।	ip addr show eth0
iptables -L -n	Firewall rules দেখাও। কোন port block আছে কিনা এখান থেকে বোঝা যাবে।	iptables -L -n -v grep DROP
tcpdump port 80	Network traffic capture করে। Service-এর মধ্যে কী data যাচ্ছে দেখা যাবে।	tcpdump -i eth0 port 80 -w capture.pcap

7. SSH -- Secure Remote Access

SSH হলো DevOps engineer-এর সবচেয়ে গুরুত্বপূর্ণ tool। Remote server manage, file transfer, tunnel -- সব SSH দিয়েই হয়।

Command	কী করে (ব্যাখ্যা)	Example
ssh user@host	Remote server-এ login করে।	ssh ubuntu@192.168.1.10
ssh -i key.pem	Specific private key দিয়ে connect করে। AWS EC2-তে .pem file দিয়ে।	ssh -i ~/.ssh/prod.pem ubuntu@ec2-ip
ssh -p 2222	Custom port-এ connect করে। Default port 22 পরিবর্তন করলে এভাবে।	ssh -p 2222 admin@myserver.com
ssh -L local:host:remote	Local port forwarding -- local port কে remote server-এর port-এ tunnel করে।	ssh -L 3306:db-server:3306 jump-server
ssh -J jump user@target	Jump host/bastion দিয়ে private server-এ access করে।	ssh -J bastion.prod ubuntu@private-server
scp file user@host:/path	File securely copy করে remote server-এ। SSH-এর উপর দিয়ে কাজ করে।	scp -i key.pem app.jar ubuntu@server:/opt/app/
ssh-keygen -t ed25519	New SSH key pair তৈরি করে। ed25519 হলো সবচেয়ে secure ও modern algorithm।	ssh-keygen -t ed25519 -C 'devops@company.com'
ssh-copy-id user@host	Public key server-এ copy করে। এরপর password ছাড়াই login হবে।	ssh-copy-id -i ~/.ssh/id_ed25519.pub ubuntu@server
~/.ssh/config	Short alias তৈরি করে। প্রতিবার ip/port টাইপ না করে short name ব্যবহার।	Host prod / HostName 192.168.1.10 / User ubuntu

8. Git -- Version Control

Git ছাড়া DevOps কল্পনাও করা যায় না। CI/CD pipeline, infrastructure as code, configuration management -- সব কিছু Git-এ থাকে।

Command	কী করে (ব্যাখ্যা)	Example
<code>git clone url</code>	Repository download করে। --depth 1 দিলে শুধু latest version আসে, দ্রুত হয়।	<code>git clone --depth 1 https://github.com/org/repo.git</code>
<code>git status</code>	কোন files changed, staged, untracked তা দেখে। কাজ শুরুর আগে সবসময় check করে।	<code>git status</code>
<code>git add -A</code>	সব changed files stage করে commit-এর জন্য।	<code>git add -A && git status</code>
<code>git commit -m</code>	Changes commit করে message সহ। Good commit message = what + why।	<code>git commit -m 'fix: resolve nginx 502 on health check'</code>
<code>git push origin main</code>	Local commits remote-এ push করে। CI/CD pipeline এখান থেকে trigger হয়।	<code>git push origin feature/new-deployment</code>
<code>git pull --rebase</code>	Remote changes নিয়ে আসে। --rebase দিলে merge commit তৈরি হয় না, cleaner।	<code>git pull --rebase origin main</code>
<code>git checkout -b</code>	নতুন branch তৈরি করে ও সেখানে switch করে। Feature সবসময় আলাদা branch-এ।	<code>git checkout -b hotfix/db-connection</code>
<code>git log --oneline</code>	Commit history সংক্ষেপে দেখে। --graph দিলে visual branch tree দেখা যায়।	<code>git log --oneline --graph --all head -20</code>
<code>git stash</code>	Current changes temporarily সরিয়ে রাখে। Emergency fix করতে কাজে লাগে।	<code>git stash && git checkout main</code>
<code>git revert HEAD</code>	Last commit undo করে। Production-এ safe rollback-এর জন্য এটাই ব্যবহার করে।	<code>git revert HEAD --no-edit</code>
<code>git tag -a v1.0.0</code>	Release tag তৈরি করে। Docker image tag ও deployment-এ এই tag ব্যবহার হয়।	<code>git tag -a v1.0.0 -m 'Release 1.0.0' && git push --tags</code>

9. Docker -- Container Management

Docker হলো modern DevOps-এর অন্যতম প্রধান tool। Application containerize করলে "works on my machine" সমস্যা চলে যায়। Production deployment সহজ ও consistent হয়।

Command	কী করে (ব্যাখ্যা)	Example
<code>docker ps</code>	Running containers দেখাও। -a দিলে stopped containers-ও দেখাবে।	<code>docker ps -a --format 'table {{.Names}} \t{{.Status}}'</code>
<code>docker run -d</code>	Container background-এ start করে। -d=detached, -p=port, -e=env, -v=volume।	<code>docker run -d -p 8080:80 --name web -e ENV=prod nginx:latest</code>
<code>docker logs -f name</code>	Container-এর logs real-time দেখাও। Error debug-এ প্রথমেই এটা check করে।	<code>docker logs -f --tail 100 myapp</code>
<code>docker exec -it sh</code>	Running container-এর ভেতরে ঢুকে। Debug করতে shell access করে।	<code>docker exec -it myapp bash</code>

docker stop / rm	Container stop করো ও মুছো। rm -f = running অবস্থায়ও force remove করো।	docker stop myapp && docker rm myapp
docker build -t name .	Dockerfile থেকে image build করো। --no-cache দিলে fresh build হবে।	docker build -t myapp:v1.0 --no-cache .
docker push image	Image registry-তে push করো। আগে docker login করতে হবে।	docker push myregistry.com/myapp:v1.0
docker inspect name	Container-এর সব detail দেখাও। IP, volumes, env vars সব পাওয়া যাবে।	docker inspect myapp jq '[0].NetworkSettings'
docker system prune	Unused containers, images মুছো। Disk space free করতে নিয়মিত চালাও।	docker system prune -af --volumes
docker-compose up -d	সব services একসাথে start করো। Multi-container app manage করার সহজ উপায়।	docker-compose up -d --build
docker stats	Real-time container resource usage। CPU, Memory, Network, Disk I/O সব।	docker stats --no-stream

10. Kubernetes (kubectl) Commands

Kubernetes হলো container orchestration platform। Large scale deployment, auto-scaling, self-healing -- এগুলো K8s ছাড়া করা কঠিন। kubectl হলো K8s-এর CLI tool।

Command	কী করে (ব্যাখ্যা)	Example
<code>kubectl get pods</code>	সব pods-এর status দেখাও। -n=namespace, -A=সব namespace, -o wide=বিস্তারিত।	<code>kubectl get pods -n production -o wide</code>
<code>kubectl get all</code>	সব resources (pods, services, deployments) একসাথে দেখাও।	<code>kubectl get all -n myapp</code>
<code>kubectl describe pod</code>	Pod-এর বিস্তারিত ও events দেখাও। CrashLoopBackOff debug করতে সবচেয়ে কাজের।	<code>kubectl describe pod myapp-pod -n prod</code>
<code>kubectl logs -f pod</code>	Pod-এর logs দেখাও। --previous = crashed pod-এর logs।	<code>kubectl logs -f myapp-pod --tail=100 -n prod</code>
<code>kubectl exec -it pod</code>	Pod-এর ভেতরে shell access করো। Debug করার জন্য container-এ ঢুকতে।	<code>kubectl exec -it myapp-pod -n prod -- bash</code>
<code>kubectl apply -f</code>	YAML file থেকে resources create বা update করো। Infrastructure as Code।	<code>kubectl apply -f deployment.yaml -n prod</code>
<code>kubectl rollout status</code>	Deployment rollout-এর progress দেখাও। CI/CD-এ deployment success check।	<code>kubectl rollout status deployment/myapp -n prod</code>
<code>kubectl rollout undo</code>	Last deployment rollback করো। Production-এ bug গেলে দ্রুত এটা চালাও।	<code>kubectl rollout undo deployment/myapp -n prod</code>
<code>kubectl scale</code>	Replica count বাড়াও বা কমাও। Traffic অনুযায়ী manually scale করতে।	<code>kubectl scale deployment myapp -- replicas=5 -n prod</code>
<code>kubectl top pods</code>	Pod-এর CPU ও Memory usage দেখাও। Resource bottleneck identify করতে।	<code>kubectl top pods -n prod --sort-by=memory</code>
<code>kubectl get events</code>	Cluster-এর recent events দেখাও। Problem diagnose করার প্রথম জায়গা।	<code>kubectl get events -n prod --sort-by=lastTimestamp</code>
<code>kubectl port-forward</code>	Local machine থেকে pod-এর port access করো। Debug বা quick test করতে।	<code>kubectl port-forward pod/myapp-pod 8080:80 -n prod</code>

11. Systemd -- Service Management

Linux-এ সব services (nginx, mysql, app) systemd দিয়ে manage হয়। Server restart হলে auto-start, crash হলে auto-restart -- সব systemd করে।

Command	কী করে (ব্যাখ্যা)	Example
<code>systemctl status nginx</code>	Service-এর current status -- running/stopped, recent logs, PID।	<code>systemctl status nginx</code>
<code>systemctl start/stop</code>	Service start বা stop করো।	<code>systemctl restart nginx</code>
<code>systemctl reload</code>	Config reload করো -- service বন্ধ না করেই। Zero downtime।	<code>systemctl reload nginx</code>

systemctl enable	Boot-এ auto-start চালু করো। Reboot-এ automatically চালু হবে।	systemctl enable nginx
systemctl list-units	সব active services দেখাও। --failed = শুধু failed services।	systemctl list-units --failed
journalctl -u svc -f	Service-এর logs real-time দেখাও।	journalctl -u nginx -f --since '2 hours ago'
journalctl -p err	শুধু error-level logs দেখাও। err, warning, crit -- priority levels।	journalctl -p err --since today
systemctl daemon-reload	নতুন unit file load করো। Custom service তৈরির পর এটা চালাতে হয়।	systemctl daemon-reload && systemctl restart myapp

12. Cron -- Task Scheduling

Cron format: `minute(0-59)` `hour(0-23)` `day(1-31)` `month(1-12)` `weekday(0-7)` `command` — * মানে সব।

Command	কী করে (ব্যাখ্যা)	Example
crontab -e	Current user-এর crontab edit করো। নতুন task যোগ করতে এই command।	crontab -e
crontab -l	Current crontab দেখাও। কোন কোন scheduled tasks চলছে confirm করো।	crontab -l
0 2 * * * cmd	প্রতিদিন রাত ২টায় run করো।	0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
* / 5 * * * * cmd	প্রতি ৫ মিনিটে run করো। Health check বা monitoring script-এর জন্য।	* / 5 * * * * curl -sf http://localhost/health systemctl restart app
0 0 * * 0 cmd	প্রতি রবিবার মধ্যরাতে run করো। Weekly cleanup বা report-এর জন্য।	0 0 * * 0 find /var/log -name '*.log' -mtime +30 -delete
@reboot cmd	Server reboot হলে automatically run করো।	@reboot /opt/app/start.sh >> /var/log/app.log 2>&1

13. Environment Variables & Shell Tricks

Passwords, API keys, database URLs -- সব env variable হিসেবে রাখো, code-এ hardcode করো না।

Command	কী করে (ব্যাখ্যা)	Example
export VAR=value	Environment variable set করো। Current shell ও child processes-এ দেখা যাবে।	export DB_URL='postgresql://user:pass@host/db'
echo \$VAR	Variable-এর মান দেখাও।	echo \$PATH; echo \$HOME; echo \$USER
env grep KEY	সব env variables দেখাও, filter করা সহ।	env sort grep -i 'db api secret'
source .env	.env file-এর variables current shell-এ load করো।	source /opt/app/.env
cmd > file 2>&1	stdout ও stderr দুটোই file-এ redirect করো। Cron job-এর output log-এ রাখতে।	deploy.sh > /var/log/deploy.log 2>&1
cmd1 && cmd2	cmd1 succeed হলে cmd2 চালাও।	make build && make test && make deploy

cmd1 cmd2	cmd1 fail হলে cmd2 চালাও।	ping -c 1 server echo "Server is down!"
alias ll='ls -la'	Command shortcut তৈরি করো। ~/.bashrc-এ রাখলে permanent হবে।	alias k='kubectl'; alias d='docker'
Ctrl+R	Reverse search -- আগের command interactive ভাবে খোঁজো। সবচেয়ে time-saving shortcut।	Ctrl+R তারপর type করো
history grep cmd	আগে চালানো command খোঁজো।	history grep 'docker run'

14. Permission Numbers -- Quick Reference

$r=\text{read}(4)$, $w=\text{write}(2)$, $x=\text{execute}(1)$ -- তিনটি যোগ করলে permission number পাওয়া যায়। যেমন: $rwx = 4+2+1 = 7$, $rw- = 4+2+0 = 6$ ।

chmod	Owner	Group	Others	কখন ব্যবহার করবে
777	rwX	rwX	rwX	কখনো না! সবাই সব করতে পারবে -- Security nightmare!
755	rwX	r-X	r-X	Web directories, shell scripts-এর জন্য standard।
644	rw-	r--	r--	Config files, web assets-এর জন্য standard।
600	rw-	---	---	SSH private keys-এর জন্য অবশ্যই এটাই ব্যবহার করো।
400	r--	---	---	AWS .pem files, read-only secret files।
750	rwX	r-X	---	Team-accessible scripts, group-এ access দিতে।
640	rw-	r--	---	Sensitive config files যা শুধু group read করতে পারবে।

Real-World DevOps Workflows

Incident Response -- Server Down হলে কী করবে

```
# Step 1: Server-এ connect করো
ssh -i prod-key.pem ubuntu@prod-server

# Step 2: System status check করো
uptime && free -h && df -h

# Step 3: Service status দেখো
systemctl status nginx app-service

# Step 4: Recent logs দেখো
journalctl -u nginx --since "30 minutes ago"
tail -n 100 /var/log/app/error.log

# Step 5: Port চালু আছে কিনা দেখো
ss -tuln | grep ':80\|:443\|:8080'

# Step 6: CPU/Memory কোনটা বেশি নিচ্ছে
ps aux --sort=-%cpu | head -10

# Step 7: Restart করো
systemctl restart nginx
```

New Application Deploy করো

```
# Step 1: Code নামাও
cd /opt/app && git pull --rebase origin main

# Step 2: Docker image build করো
docker build -t myapp:$(git rev-parse --short HEAD) .

# Step 3: Deploy করো
docker-compose down && docker-compose up -d
```

```
# Step 4: Health check করো
sleep 5
curl -f http://localhost/health && echo "Deploy successful!" || echo "Deploy FAILED!"

# Step 5: Logs দেখো
docker-compose logs -f --tail=50
```

Disk Full হলে কী করবে

```
# Step 1: কোথায় জায়গা নেই
df -h

# Step 2: বড় folder খোঁজো
du -sh /var/log/* | sort -h | tail -10

# Step 3: বড় files খোঁজো
find / -size +500M -type f 2>/dev/null

# Step 4: পুরনো logs মুছো
find /var/log -name '*.log' -mtime +30 -delete

# Step 5: Docker cleanup করো
docker system prune -af
```

Production Safety Rules: (1) `rm -rf` চালানোর আগে path double-check করো | (2) Database-এ কিছু করার আগে backup নাও | (3) `.env` file কখনো git-এ push করো না | (4) staging-এ test করো, তারপর production-এ নাও

এই *cheat sheet* print করো ও *monitor*-এর কাছে রাখো। *Commands* মুখস্থ করার চেয়ে বারবার ব্যবহার করলেই মনে থাকবে। *Practice is the key!*

github.com/YOUR_USERNAME/devops-linux-cheatsheet